# GreenMind LLM Inference Optimization Hackathon



CMKL UNIVERSITY

AIE AI ENGINEERING INSTITUTE

ExxonMobil

# Everyone optimizes accuracy, throughput and latency. Almost no one optimizes power

**What most LLM inference work tracks**

### Accuracy

Does the model still answer correctly?

### Throughput

How many requests or tokens per second can it serve?

### Latency

How quickly does the answer start and finish?

### Power consumption

How many joules does each answer cost? Rarely measured, rarely tuned.

*usually ignored*

### Why this hackathon

GreenMind puts energy on the scoreboard next to accuracy, throughput and latency. You win by serving correct, on-time answers with the fewest joules, so efficiency becomes a first-class engineering goal instead of an afterthought.

# Your job: make MANGO1.5-Qwen3.5-9B serve more on-time work per joule

## The problem

- Start and optimize a local LLM inference server for the organizer-provided model
- You are free to change the model and the serving stack
- The server must stay compatible with the grader's request and response contract: OpenAI-compatible chat completions with streaming

**CMKL/MANGO1.5-Qwen3.5-9B**

The base model, in BF16. The untuned baseline serves this with default vllm serve.

**CMKL/MANGO1.5-Qwen3.5-9B-AWQ-W4A16-mtp**

A quantized variant that includes a multi-token-prediction head.

**Hybrid Mamba / Gated-DeltaNet**

Only about 8 of 32 layers use a conventional KV cache, so long-context memory behaves differently from a plain transformer.

# Every team gets the same dedicated A100 40GB, so results are comparable

**1x**

NVIDIA A100-SXM4 40 GB, dedicated by PCI passthrough

**16**

vCPU, AMD EPYC 7742

**48 GB**

RAM (47 GiB usable)

**~145 GB**

root disk, about 110 GB free at hand-out

### Software and access

- Ubuntu 24.04 LTS,Docker, NVIDIA driver 580
- User and Password will provide

# Two rounds decide the winner: automated scoring, then pitching

### Round 1

**Technical submission**

The grader replays a hidden trace against your endpoint and computes your score S.

### Top 8

**advance**

Ranked by best Final score. Ties go to higher accuracy retention, then lower p95 TTFT.

### Round 2

**Pitching**

7 minutes of presentation, then 7 minutes of Q&A with the judging panel.

### Final ranking

**40 / 60**

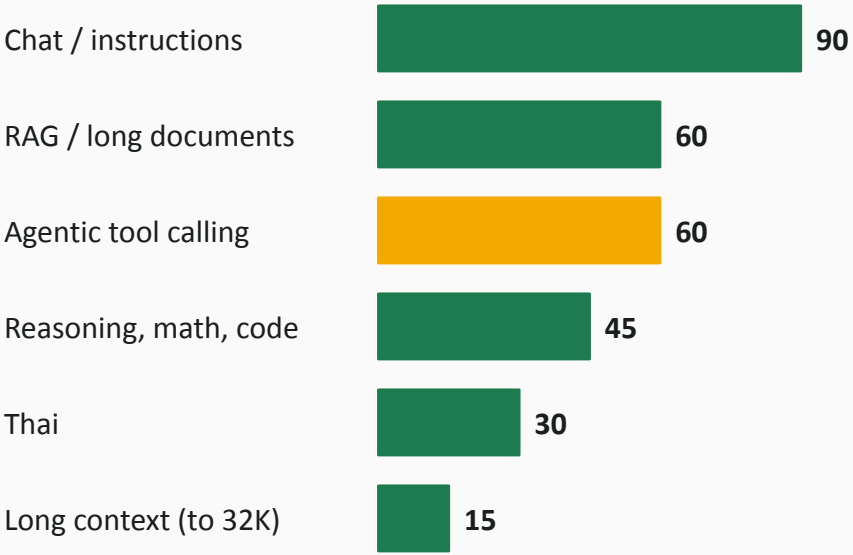
Round 1 counts 40%, Round 2 counts 60%. Both are on a 0 to 100 scale.

## Final score = 0.4 x Round 1 + 0.6 x Round 2

Teams outside the top 8 do not pitch and are not ranked in the final standings.

# The hidden test set mirrors a 2026 production server

## Final track: 300 requests by class




## Load shape

1. **Ramp:** request rate rises form (0 - 4 req/s)
2. **Peak:** constant high rate; this phase separates good servers from weak ones (4 req/s)
3. **Quiet:** low rate; idle power shows (1 req/s)

- Open loop: requests arrive on schedule whether or not earlier ones finished
- Every request: thinking mode on, max_tokens 8192, temperature 0
- Some are multi-turn tool-calling episodes, up to 15 turns

# A request is useful work only if it meets all five Service Level Objective (SLO) conditions

1. It streamed back, so time to first token (TTFT) can be observed
2. No error or timeout (900 s)
3. Not cut off at the 8,192-token limit
4. Time to first token (TTFT) <= 0.5 s + 0.25 s per 1,000 prompt tokens
5. Time per output token (TPOT) <= 50 ms (skipped for one-token answers)

*Late, failed or cut-off requests still burn energy but add no work.*

**TTFT allowance grows with prompt length**

**0.575 s**
300-token chat

**2.0 s**
6,000-token RAG question

**5.5 s**
20,000-token long context

# One score decides Round 1: accuracy gates it, energy drives it

$$S = 100 \cdot A \cdot s_E^{0.6} \cdot s_T^{0.2} \cdot s_L^{0.2}$$

## A

**Accuracy factor**

Range 0.05 to 1. Are you still as correct as the baseline? Multiplies everything.

## s_E

**Energy score**

Weight 0.6. On-time work per joule versus the baseline.

## s_T

**Throughput score**

Weight 0.2. Share of peak-phase requests served on time.

## s_L

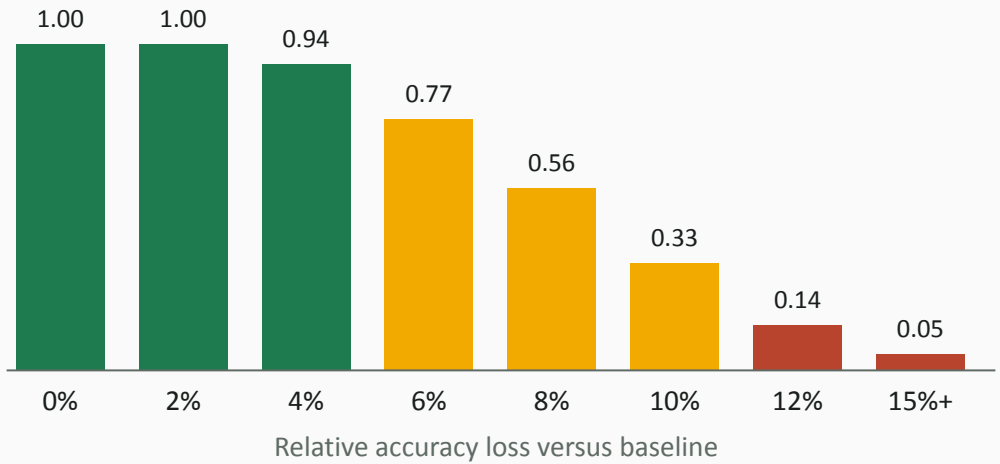
**Latency score**

Weight 0.2. How close your p95 TTFT and TPOT are to the targets.

Factors multiply, so if any one is 0, S is 0. The untuned baseline (default vllm serve, BF16 MANGO, same VM type) scores S = 0 by definition: you score by beating it.

# Accuracy multiplies everything: a 2% loss is free, a 10% loss costs two thirds

**Accuracy factor A by relative accuracy loss (class factor = 1)**



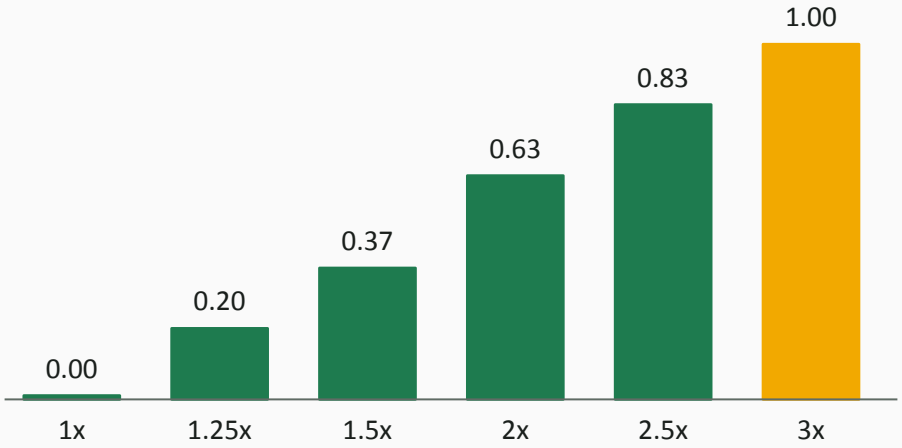

- R = your macro-accuracy ÷ baseline macro-accuracy, averaged over six classes
- R of 0.98 or more gives A = 1; R of 0.85 or less gives the floor of 0.05
- Class check: a class with at least 20 questions that keeps under 75% of the baseline's accuracy multiplies A by class_factor, from 1.0 at 75% down to 0 at 50%. The weakest class counts. Example: weakest class at 62.5% halves A
- Beating the baseline earns no extra credit

$$A = \max(0.05, \text{smoothstep}(x) \times \text{class_factor}), \quad x = \text{clip}((R - 0.85) \div 0.13, 0, 1), \quad \text{smoothstep}(x) = 3x^2 - 2x^3$$
$$\text{class_factor} = \text{smoothstep}(\text{clip}((c - 0.50) \div 0.25, 0, 1)), \quad c = \text{weakest class's accuracy} \div \text{the baseline's on that class}$$

# Energy carries 60% of the weight: efficiency is on-time requests per joule

- η = SLO on-time requests ÷ GPU joules over the whole timed window, idle gaps and drain included
- Every doubling of efficiency earns the same points, because the score is logarithmic
- K = 3 on both Validation and Final: three times the baseline's efficiency is full marks

**s_E by efficiency versus the baseline (K = 3)**




Your efficiency (on-time requests per joule) as a multiple of the baseline's

**s_E = clip( ln( η ÷ η_baseline ) ÷ ln K , 0, 1 ),  η = on-time requests ÷ total GPU joules,  K = 3**

# Throughput and latency stop paying at 1.0, so speed pays through energy

## Throughput s_T

**s_T = (peak-phase requests that passed SLO) ÷ (peak-phase requests that arrived)**

Only the constant-high-rate peak phase counts. A request that is late, failed or cut off counts as not served.

## Latency s_L

**$s_L = \sqrt{( s_{L,ttft} \times s_{L,tpot} )}$**
**$s_{L,ttft} = \min( 1 , 1 \div p95( TTFT \div its\ SLO ) )$**
**$s_{L,tpot} = \min( 1 , 50\ ms \div p95( TPOT ) )$**

p95 means the slowest 5% of requests set your score, so a bad tail is penalised.

**Inside the targets, going faster earns nothing directly. It pays only through lower energy: fewer GPU-seconds for the same work.**

# Validation is practice; only the Final run is graded

| | Validation | Final |
|---|---|---|
| Purpose | Practice: iterate, debug, tune | The graded result |
| Requests | 100 | 300 |
| Rounds | 1 | 1 |
| Submissions allowed | 5 per hour | 2 per hour |
| Score | Indicative only | Counts for ranking |

Your score on a track is your best finished submission. Failed, running or interrupted submissions are not ranked. Use Validation to check that your configuration is stable before you spend a Final submission.

Grader submissions close at Day 2 (Oct 25th) 11:00 am.

# Rules (1 of 2): derive from MANGO and use open source only

## Model

- Serve CMKL/MANGO1.5-Qwen3.5-9B or a model derived from it
- Allowed: quantization, pruning, calibration, weight conversion, adding or removing speculative-decoding heads such as MTP
- Fine-tuning and distillation are allowed, on open data only
- Derived weights must be made by you during the event, from the official MANGO weights
- A speculative-decoding draft model may be any open-source model, disclosed. Its energy counts

## Software and data

- Engines, frameworks, tools, libraries and datasets must be open source: vLLM, SGLang, TensorRT-LLM, llama.cpp, your own kernels
- No closed-source binaries, hosted APIs or proprietary services
- Calibration and compression datasets must be open and must not contain hidden test questions
- Custom kernels, schedulers and serving code are welcome. Source must be available to organizers on request

# Rules (2 of 2): serve from your own VM, hold power settings, play fair

## Serving

- OpenAI-compatible /chat/completions with streaming and token usage
- Tool calling must work (tool_choice auto)
- The grader fixes max_tokens, temperature, thinking mode and sampling
- Serve from your own VM and its single GPU
- No response caching of graded requests; normal KV and prefix caching is fine

## Hardware and runtime

- A fixed power limit or locked clocks are allowed if held constant for the whole run
- No settings that apply only during the measured window
- No tampering with the measurement: no LD_PRELOAD, no replacing the NVIDIA management library (nvml)
- One serving process, no other GPU workloads and no external calls during a graded run

## Fair play

- No collusion or sharing of submissions and credentials
- Do not try to obtain or reconstruct hidden test items
- Do not touch other teams' VMs or the grader host (hacking,DDos)
- Organizers may re-run any submission and inspect your configuration
- Violations may lead to disqualification

# Submit through the grader in four steps

1

### Get the server up

Streaming, token usage and tool calling enabled, reachable at http://<vm-ip>:<port>/v1.

```
--enable-auto-tool-choice
--tool-call-parser
qwen3_coder
```

2

### Log in, change display name,save endpoint

Use the username and password from the organizers and change the password and display team name. In Settings, save the endpoint and pick your model from the dropdown.

3

### Try first submit

Run Evaluation: choose Validation, click Health check (chat and GPU energy counter), then Submit. Results appear in History.

4

### Start optimizing your server

Research and apply the optimization techniques to your endpoint and submit

**The grader address and team credentials are announced by the organizers.**

# Top 8 teams pitch for 7 minutes, then defend their choices for 7

## 40%

### Technical approach and trade-offs

Why it worked, what you gave up, and the attempts that failed.

## 40%

### Sustainability impact

Does it matter beyond the leaderboard and hold up at production scale?

## 20%

### Clarity and delivery

Structure within the time, your numbers versus baseline, and Q&A.

### Before pitching, the top 8 submit (by the 11:30 am. on day 2 Oct 25)

A GitHub link to your experiments: serving commands, configs, code and results, including what did not work with explanation

Hugging Face links to every optimized weight you used, or a statement that you used only the originals

Presentation Slide as pdf file.

# Let’s Start hackathon

**Grader :** GreenMind Hackathon

**Submit From :** From

**Registration Form:** From

# Please join our Discord Channel for easier communication



https://discord.gg/Wvy2SwZxKh

# Questions?

Measure, iterate, and be ready to explain your trade-offs.

